❶ and dereference that pointer ❷. In the meantime, another thread could update the pointer ❸, eventually leading to the node being deallocated (in `pop()`). If the node is deallocated before you dereference the pointer, you have undefined behavior. Ouch! It's tempting to add an external count in `tail` the same as you did for `head`, but each node already has an external count in the `next` pointer of the previous node in the queue. Having two external counts for the same node requires a modification to the reference-counting scheme to avoid deleting the node too early. You can address this by also counting the number of external counters inside the `node` structure and decreasing this number when each external counter is destroyed (as well as adding the corresponding external count to the internal count). If the internal count is zero and there are no external counters, you know the node can safely be deleted. This is a technique I first encountered through Joe Seigh's Atomic Ptr Plus Project.[5] The following listing shows how `push()` looks under this scheme.

[5] Atomic Ptr Plus Project, http://atomic-ptr-plus.sourceforge.net/.

Listing 7.15. Implementing `push()` for a lock-free queue with a reference-counted tail

```
template<typename T>
class lock_free_queue
{
private:
    struct node;

    struct counted_node_ptr
    {
        int external_count;
        node* ptr;
    };

    std::atomic<counted_node_ptr> head;
    std::atomic<counted_node_ptr> tail;     <-(1)

    struct node_counter
    {
        unsigned internal_count:30;
        unsigned external_counters:2;       <-(2)
```